FTD Master

AI-Powered ESG Analysis

Session 2 – RAG: HyDE, MQR & Advanced Retrieval

Theory & Hands-on Application

Rami El Mnebhi

April 2026

UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

Why Do We Need RAG?

LLMs are powerful, but they forget one thing: your data.

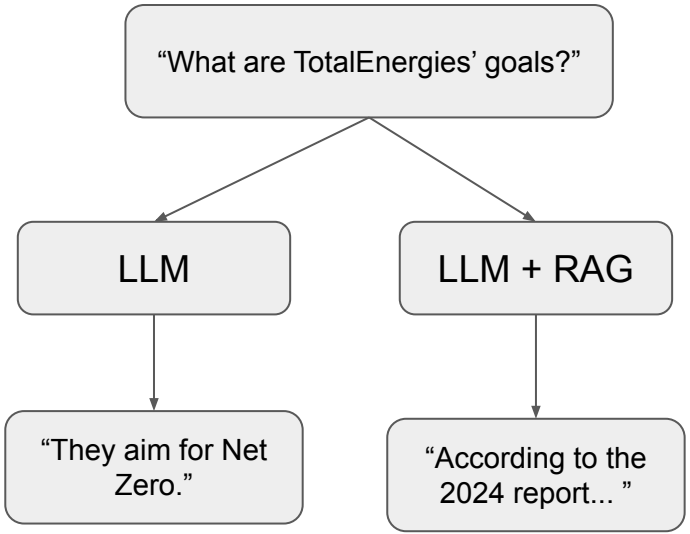
Without access to private or external documents, an LLM will:

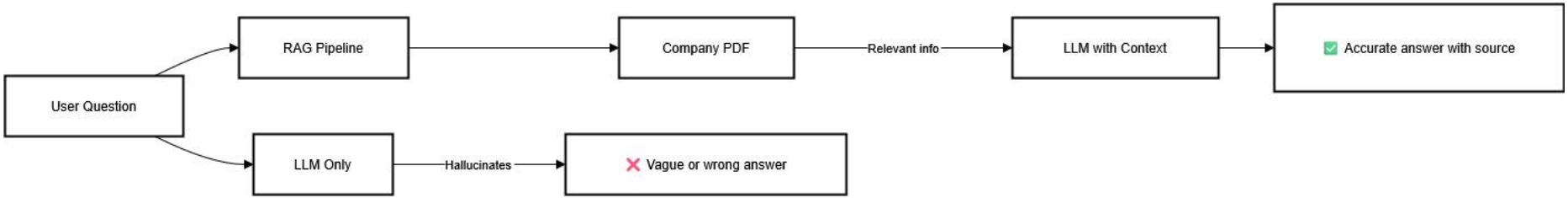
- Hallucinate facts
- Confuse companies or timeframes
- Give vague or generic answers

Example:

❓ "What are TotalEnergies' carbon reduction goals for 2030?"

→ A generic LLM will hallucinate — unless we give it the source text.





What is Retrieval-Augmented Generation (RAG)

RAG combines search with generation.

Instead of asking an LLM to "remember everything," we help it by:

- Retrieving relevant document chunks
- Injecting them into the LLM's prompt
- Letting it generate an informed, contextual answer

This makes the model:

🔍 More accurate
💬 More grounded
✅ More trustworthy for real-world tasks

RAG Architecture: Main Components

A RAG pipeline is made of modular parts — each one affects the final result.
Here's what we build and customize:

📦 <u>Components Overview:</u>

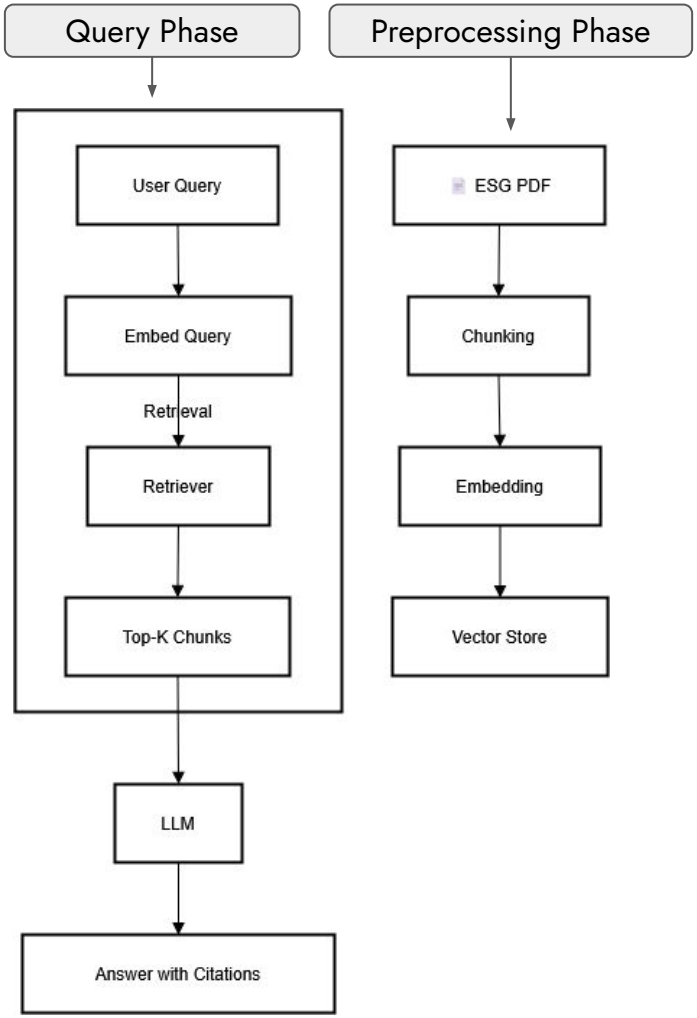
🧩 Chunking
- Split large documents into smaller, manageable text units
- Too big = irrelevant, too small = fragmented

🧠 Embedding
- Convert chunks into vectors using a model (e.g., Albert API embeddings)

🗂️ Vector Store
- Store & index embeddings for fast similarity search (FAISS, Chroma, OpenSearch)

🔍 Retriever
- For each user query, find the top-K most similar chunks

🤖 LLM
- Takes the question + retrieved context and generates an answer



Building a Mini RAG Pipeline (Hands-On)

Let's walk through a simple Retrieval-Augmented Generation pipeline.
You'll use your PDF content, convert it to vector chunks, and generate ESG answers.

🧪 Steps Overview

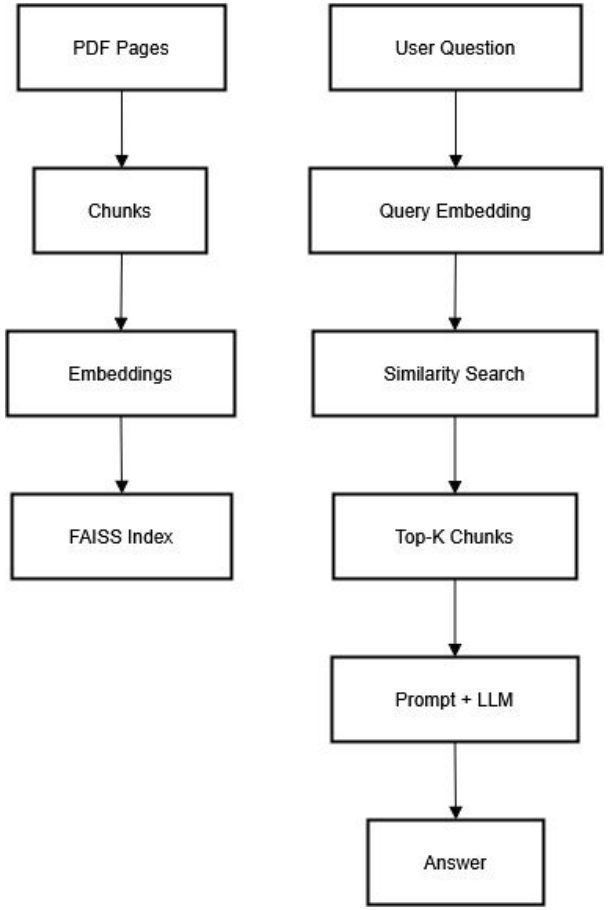
1. Chunk the PDF
- Use a parser like PyMuPDF or pdfplumber to extract clean text

1. Embed the Chunks
- Use Albert API's BGE-M3 to create embeddings for each chunk

1. Store in a Vector Database
- Use FAISS or Chroma to index your documents

1. Ask a Question
- Embed the query and retrieve relevant chunks

1. Send to LLM
- Inject top chunks into prompt + generate the final answer



Common Pitfalls in RAG (and How to Avoid Them)

Even simple RAG works — but many issues arise when scaling or dealing with real documents.

❗ Pitfall #1: Bad Chunking

📉 Too long: irrelevant fluff

🧩 Too short: missing context

Solution: **Tune chunk size (~300–500 tokens), use overlap if needed**

❗ Pitfall #2: Weak Retrieval

🔍 Returns irrelevant chunks

🤖 LLM has no useful context to work with

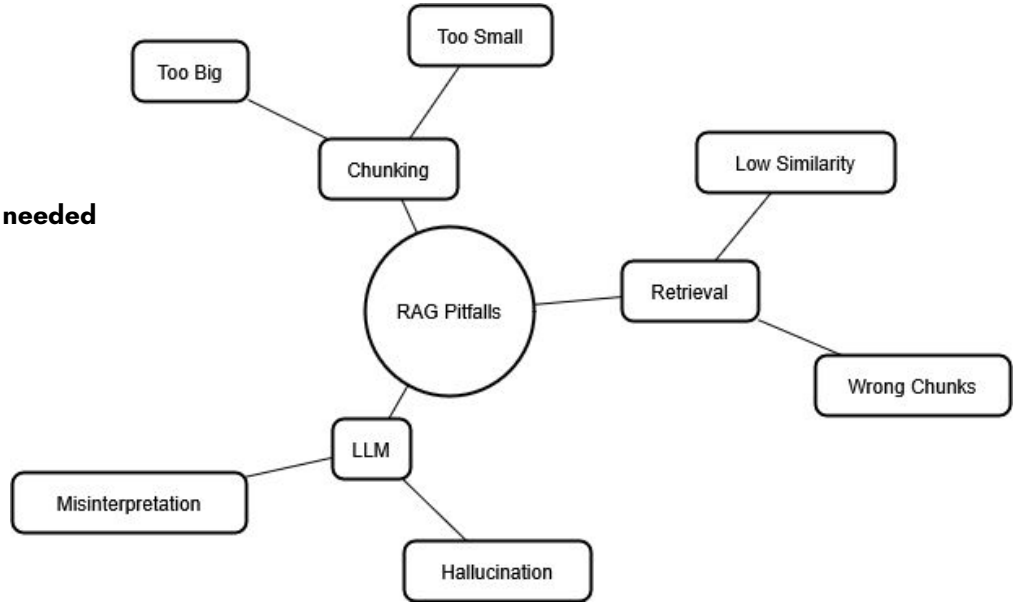
Solution: **Use better embedder, refine query, increase top-k**

❗ Pitfall #3: LLM Hallucination

🫠 Still makes things up if context is weak

🧪 Answers might sound plausible but be factually wrong

Solution: **Use citations, show sources, evaluate outputs (ROUGE, RAGAS)**



What You'll Build in This Session

You'll implement your first end-to-end RAG pipeline using your team's ESG report.

🛠️ Tasks for Today:

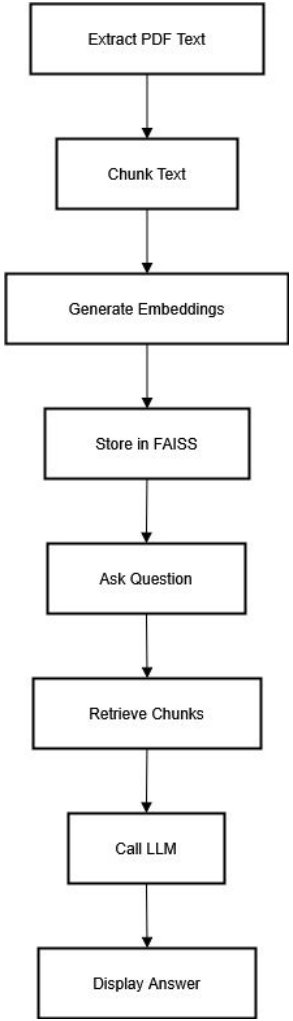
- ✅ Extract text from your assigned PDF
- ✂️ Chunk it into sections (~300–500 tokens)
- 🧠 Embed the chunks using Albert API's BGE-M3
- 📦 Store embeddings in FAISS (or in-memory if needed)
- ❓ Ask ESG-related questions using the dataset
- 💬 Use LLM + retrieved chunks to generate contextual answers

🎯 Goals by the end of the session:

Have a working RAG notebook / scrip / app

See meaningful answers grounded in your actual documents

Identify where retrieval works — and where it fails



Stretch Goals & What's Next in RAG

🛠️ If You Finish Early… Try This:

🔁 Experiment with Chunking Strategies

- Use overlapping chunks
- Try sentence-based splitting with nltk or spacy

🧪 Swap Embedders

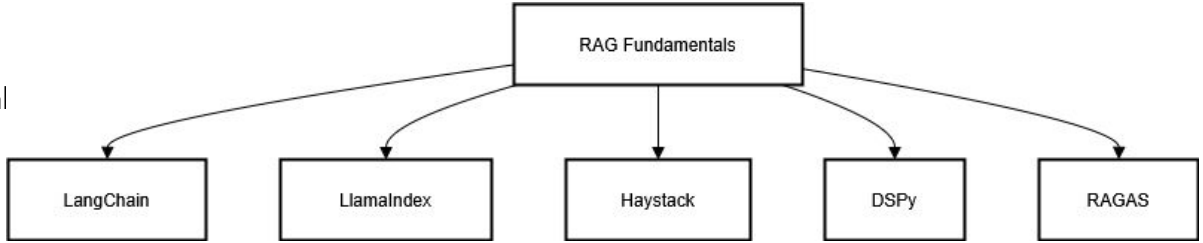
⚙️ Add Re-Ranking Logic

- Use cosine similarity to sort chunks post-retrieval
- Optional: try rerankers from llama-index or Haystack

📊 Evaluate Answers Automatically

- Use RAGAS or ROUGE to measure output qual

Framework	Strengths
LangChain	High-level abstraction, great for chaining
LlamaIndex	Document-centric, powerful with graphs
Haystack	Open-source German framework, scalable
DSPy	Optimization-oriented RAG pipelines
RAGAS	Evaluate factuality and citation accuracy



Beyond Naive RAG: Hypothetical Document Embedding (HyDE)

🔍 The Problem with Naive RAG Queries

Raw user queries are short and vague — embedding space mismatch with document chunks

💡 HyDE: The Idea

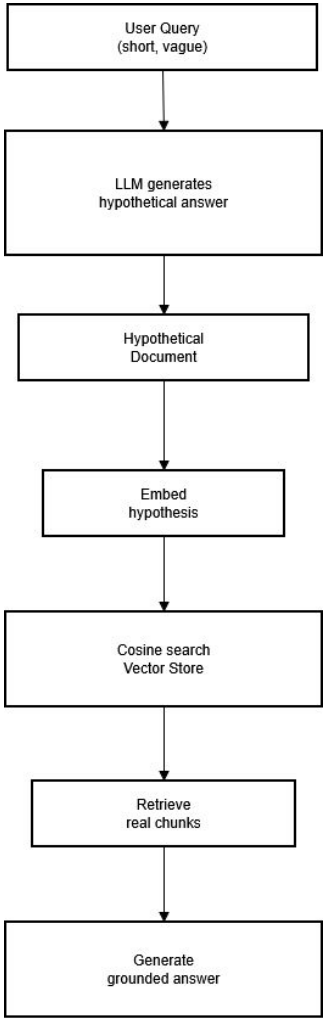
Instead of embedding the raw query, ask the LLM to generate a hypothetical answer first

Then embed THAT hypothetical document and use it to retrieve real chunks

⚙️ HyDE Pipeline

Query → LLM generates hypothetical doc → embed hypothetical doc → cosine search → retrieve real chunks → generate answer

📈 When does it help? Especially on complex, technical queries where vocabulary mismatch is high



Multi-Query Retrieval (MQR): Beating the Single-Query Bottleneck

🎯 The Problem with a Single Query

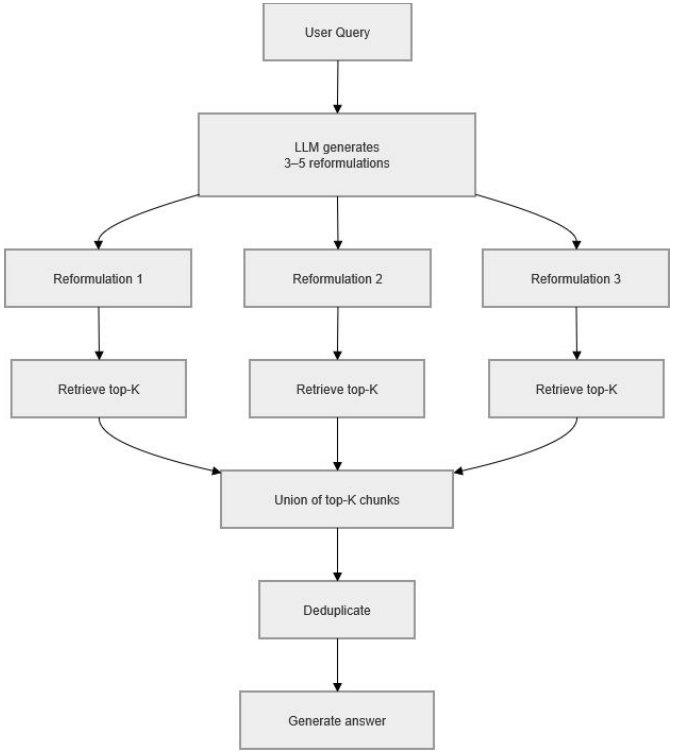
One query = one angle of retrieval. Real questions have multiple facets.

💡 MQR: The Idea

Use the LLM to expand one user query into N semantically diverse sub-queries

Run retrieval for EACH sub-query, then de-duplicate and merge the results

⚙️ MQR Pipeline

Query → LLM generates 3–5 reformulations → parallel retrieval → union of top-K chunks → deduplicate → generate answer

🔧 In LlamaIndex: MultiStepQueryEngine / SubQuestionQueryEngine. In LangChain: MultiQueryRetriever